

Uno Systems Lab

Bare-metal Rust, robotics runtime, and GPU systems roadmap

North-star objective

Build an open-source system spanning bare-metal device firmware, a versioned sensor/actuator protocol, a Rust Linux runtime, robotics control, networking, and eventually CUDA-accelerated perception on NVIDIA Jetson.

Status snapshot

As of	August 30, 2026
Current hardware	Elegoo Uno R3 / ATmega328P, onboard LED, tactile button, USB serial
Firmware	Rust <code>no_std</code> / <code>no_main</code> with custom AVR startup, linker layout, interrupts, and HAL-style modules
Current demo	Debounced button cycles Heartbeat / Solid On / Off and reports mode changes over USART-to-USB
Next milestone	Rust host application that opens the serial device, parses events, logs them, and reconnects

System architecture

Uno sensors and actuators -> ATmega328P bare-metal firmware -> USART/USB -> Rust host runtime -> Linux robotics runtime -> networking and CUDA perception -> physical rover.

Completed: bare-metal foundation

- [x] Created a `no_std`, `no_main` Rust firmware binary targeting `avr-none` / `atmega328p`.
- [x] Hand-wrote the AVR reset vector and startup entry in `startup.S`.
- [x] Initialized `r1`, `SREG`, and the stack pointer at `RAMEND 0x08FF`.
- [x] Created a linker script with 32 KB flash at `0x000000` and 2 KB SRAM at `0x800100`.
- [x] Implemented copying initialized `.data` from flash to SRAM.
- [x] Implemented zeroing `.bss` before entering Rust.
- [x] Assembled, linked, converted ELF to Intel HEX, flashed with `avrdude`, and verified disassembly.
- [x] Confirmed direct control of `PB5` / Arduino `D13` / onboard L LED.

Completed: reusable firmware library

- [x] Split the Cargo package into a reusable **src/lib.rs** library and a firmware **src/main.rs** binary.
- [x] Created **gpio.rs** with safe input/output pin APIs and encapsulated volatile register access.
- [x] Mapped usable Uno Port B pins PB0-PB5 and modeled Input / Output modes.
- [x] Added floating and internal pull-up input configuration.
- [x] Created **timer.rs** with Timer1 CTC configuration, compare interrupt enable, and prescaler selection.
- [x] Configured a 10 ms Timer1 tick: 16 MHz / 64 prescaler, OCR1A = 2,499.
- [x] Created **interrupts.rs** with a documented unsafe boundary for global interrupt enable.
- [x] Added Timer1 Compare A to the AVR vector table and an assembly ISR wrapper that preserves CPU state.
- [x] Created **debounce.rs** with configurable consecutive-sample debouncing.
- [x] Used three 10 ms samples for approximately 30 ms button debounce.
- [x] Created **serial.rs** with blocking USART0 transmit at configurable baud and 8N1 framing.
- [x] Added an AVR LPM program-memory reader so string literals can be transmitted from flash.

Completed: application behavior

- [x] Mounted a four-pin tactile switch on a breadboard between D12 / PB4 and GND.
- [x] Configured PB4 as an active-low input using the ATmega328P internal pull-up.
- [x] Implemented an interrupt-driven two-pulse heartbeat: 100 ms on, 100 ms off, 100 ms on, 700 ms off.
- [x] Implemented a three-mode state machine: Heartbeat -> Solid On -> Off -> Heartbeat.
- [x] Converted raw button levels into one debounced press event per physical press.
- [x] Kept slow serial transmission out of the ISR; the main loop reports observed mode changes.
- [x] Verified USART0 output through the Uno USB bridge at 9,600 baud.
- [x] Reported BOOT and MODE changes to a macOS serial terminal.

Known constraints and technical debt

- [] Complete the remaining ATmega328P interrupt-vector entries before enabling additional interrupt sources.
- [] Add unique peripheral ownership (for example, Pins::take) if the library becomes multi-user; v0.1 leaves conflicting assignments to the programmer.
- [] Add critical-section or atomic read-modify-write protection where register updates may occur from both main code and interrupts.
- [] Add unit tests for pure state machines, debouncing, framing, and protocol parsing.
- [] Document flash-string versus SRAM-string behavior and formalize a program-memory string type or macro.
- [] Replace ad hoc build commands with a reproducible task runner or build script after the manual toolchain is fully understood.

Roadmap and acceptance checklist

Milestone 1 - Rust host CLI

- [] Create a separate host crate that runs on macOS and Linux.
- [] Open a selected `/dev/cu.*` or `/dev/tty*` serial device at the configured baud rate.
- [] Parse BOOT and MODE text events and timestamp them.
- [] Reconnect cleanly after unplug/replug or device reset.
- [] Log received events and communication errors.

Milestone 2 - Versioned binary protocol

- [] Create a `no_std` protocol crate shared by firmware and host.
- [] Define frame fields: magic, version, message type, sequence, length, payload, CRC.
- [] Handle partial frames, garbage bytes, unknown versions, corruption, and timeouts.
- [] Add DEVICE_INFO, HEARTBEAT, SENSOR_SAMPLE, SET_OUTPUT, and ERROR messages.
- [] Publish protocol documentation and automated encoder/decoder tests.

Milestone 3 - Sensor node

- [] Inventory the Elegoo sensor kit and choose one analog sensor plus one digital bus sensor.
- [] Implement the ATmega328P ADC abstraction and calibrated analog sampling.
- [] Implement one I2C or SPI sensor driver from its datasheet without an Arduino library.
- [] Schedule sensors at independent rates and transmit samples through the binary protocol.
- [] Measure sample rate, dropped messages, latency, and error counts.

Milestone 4 - Actuation and control

- [] Add PWM/servo output, then motor-driver and encoder interfaces.
- [] Build a cooperative task scheduler with bounded work per tick.
- [] Add command validation, timeouts, safe startup outputs, and failsafe behavior.
- [] Demonstrate closed-loop speed or position control.

Milestone 5 - Linux robotics runtime

- [] Run the Rust host on a Raspberry Pi or other Linux computer.
- [] Introduce typed topics, bounded queues, backpressure, logging, and live metrics.
- [] Benchmark latency, jitter, allocations, and message loss.
- [] Add simulated devices so most development does not require physical hardware.

Milestone 6 - Rover and networking

- [] Build a rover with motor control, encoders, IMU, distance sensing, and safety input.
- [] Add UDP or QUIC transport and network discovery between robotics nodes.
- [] Implement remote telemetry and control with explicit loss and timeout behavior.

Milestone 7 - Jetson and CUDA

- [] Use a Jetson Orin Nano Super Developer Kit when camera/GPU work begins.
- [] Capture camera frames and implement CUDA grayscale, blur, and edge-detection kernels.
- [] Profile memory transfers, occupancy, coalescing, streams, and CPU/GPU synchronization.
- [] Feed GPU perception results into the Rust runtime and rover control loop.
- [] Measure end-to-end camera-to-control latency and minimize unnecessary copies.

Recovery reference

Expected project files

```
Cargo.toml
rust-toolchain.toml
.cargo/config.toml
linker.ld
startup.S
src/lib.rs
src/main.rs
src/gpio.rs
src/timer.rs
src/interrupts.rs
src/debounce.rs
src/serial.rs
```

Build and flash

```
# Reassemble startup code
avr-gcc -mmcu=atmega328p -c startup.S -o startup.o

# Clean when linker/startup behavior changes
cargo clean

# Compile and link the firmware binary
RUSTFLAGS="-C target-cpu=atmega328p" cargo +nightly rustc \
  -Z build-std=core --target avr-none --release \
  --bin uno-systems-lab -- \
  -C linker=avr-gcc \
  -C link-arg=-mmcu=atmega328p \
  -C link-arg=-nostartfiles \
  -C link-arg=startup.o \
  -C link-arg=-Wl,-Tlinker.ld

# Convert ELF to Intel HEX
avr-objcopy -O ihex -R .eeprom \
  target/avr-none/release/uno-systems-lab.elf \
  uno-systems-lab.hex

# Find the current macOS port
ls /dev/cu.*

# Flash; replace PORT with the current device
avrdude -p atmega328p -c arduino -P PORT -b 115200 -D \
  -U flash:w:uno-systems-lab.hex:i
```

Serial monitoring

```
screen PORT 9600
```

Exit screen with Ctrl-A, then backslash, then y. Close the terminal before flashing because avrdude needs exclusive access to the port.

Verification commands

```
avr-objdump -h target/avr-none/release/uno-systems-lab.elf
avr-objdump -d target/avr-none/release/uno-systems-lab.elf
avr-nm -n target/avr-none/release/uno-systems-lab.elf
```

Resume point

The next implementation task is a Rust host CLI for serial connection, event parsing, timestamps, logging, and reconnect behavior. Preserve the current text protocol until the connection lifecycle works; then introduce the shared binary protocol crate.